

Documentation in Systems Development: A Significant Criterion for Project Success

M. Faisal Fariduddin Attar Nasution
Virginia Commonwealth University
nasutionmf@vcu.edu

H. Roland Weistroffer
Virginia Commonwealth University
hrweistr@vcu.edu

Abstract

Agile software development methods seem inherently suitable for today's quick-paced business environment as they shorten the time to develop new systems and typically incur lower development costs compared to the conventional systems development life cycle (SDLC) approach. Software development project failures using conventional SDLC are often attributed to project delays, resulting in budget overruns. On the other hand, a well planned and documented systems development project is more likely to result in a system that meets the expectations of both the intended users and the software engineers. This paper takes another look at conventional SDLC methodology by focusing on an aspect that is often overlooked in systems development practice, namely the significance of good documentation.

1. Introduction

The success of a software application package can generally be traced back to its development: a development project that was both cost and time efficient. A good software project should progress on time and within budget, and should result in all of the agreed functionality [15]. The conventional systems development life cycle (SDLC) approach, however, is now often perceived as taking too long, being too expensive, and to produce software that often does not meet the users' needs and thus results in users' dissatisfaction with the software product. The successful outcome of a software development project in general depends on 1) the delivered system meeting the functional requirements of customers/users, 2) the project being completed within budget, 3) the project being completed on time, 4) the project being completed in its entirety, 5) the delivered software consisting of solid and, thoroughly tested code; and 6) the system being easy to use [20].

In contrast to the conventional SDLC approach, agile methods generally proceed with system construction without waiting for "formal requirements

analysis and design phases" [3]. Agile methods seek "to strip away as much of the heaviness, commonly associated with traditional software-development methodologies, as possible to promote quick response to changing environments, changes in user requirements, accelerated project deadlines, and the like" [7]. These methods also play down the need for formal documentation, such as well-documented software architecture and detailed project planning [25]. These characteristics of agile methods make them suitable for a quick-paced and dynamic business environment and offer rapid information systems solutions [3,17]. However, this may also results in difficulty of reusing agile developed components due to the rather loose structure of the agile approach [16]. And most importantly, omitting documenting the development process may lead to corporate memory loss [23].

The conventional SDLC approach has evolved into several newer methodologies that include techniques such as prototyping and object-oriented analysis and design. But the basic approach still consists of multiple, sequential phases that constitute specific activities for designing and developing information systems. Each phase involves designated tasks, which when followed properly should lead to an information system that meets the needs of the users [10]. The literature has noted various software development failures, which may be attributed to a lack of properly following SDLC practice.

Given these facts, can good documentation reduce the likelihood of failure that has been haunting the outcome of many conventional SDLC projects? This study seeks to investigate the impact of good documentation practice on the outcome of conventional SDLC. This study therefore looks at the documentation framework in SDLC that should deliver successful system products. The argument made is that well documented systems development assures system products that satisfy both the intended users and the software engineers. A well-documented SDLC execution that delivers satisfactory outcomes is likely

if communication and requirements elicitation practices encourage good documentation. This study is based on an extensive literatures review and a reexamination of a published case study.

1.1. Systems Development Life Cycle

The systems development life cycle methodology, or SDLC, was developed as a means to structure and improve the management of systems development through the identification of phases and activities [1]. During the early days of computer software development, SDLC was focused on programming and solving technical issues, due to the limited capability of computer hardware at that time. SDLC typically consists of various activities that make up each phase. These activities dictate what needs to be achieved and what tools should be used. The phases typically include feasibility study, systems investigation, analysis, design, development, implementation, testing, and maintenance.

The early SDLC is also referred to as the waterfall model as each phase produces an artifact that leads into the next phase, in one direction only without any going back, like water flowing over a waterfall [22]. This model is possibly the most widely known model and, with variations, used to be the most common model in industry practice. The waterfall model ideally requires that each phase be documented in a report and be validated before advancing to the next phase [13]. As such, the waterfall model is very suitable for systems development projects with well defined requirements. However, because the waterfall model does not allow for iterative creation of artifacts, any mistakes made in any phase are propagated into subsequent phases, and fixing these mistakes becomes difficult and expensive, often leading to late delivery and budget overruns.

Prototyping is an approach that, if used in conjunction with a modified SDLC model, may alleviate some of these disadvantages. Its primary use is “to reduce risk by ‘buying knowledge’ and thus to increase the likelihood of success of a software project” [4]. Prototyping provides better control than the pure waterfall model in terms of meeting user requirements.

1.2. Documenting the SDLC

Extensive documentation in SDLC plays an essential role in communicating between developers, managers, and users, particularly in the analysis phase [9]. Good documentation “allows companies to measure current performance, identify areas of weakness, and initiate improvement actions” [5].

A study by Luqi et al. (2004) denotes the benefits of document driven development (DDD), which demands comprehensive systems documentation that records all phases of SDLC and is capable of handling even complicated issues often related to real-time systems. DDD enables users, or stakeholders, to effectively monitor the systems development process and to communicate with each other by providing users the opportunity to receive the compiled information [11]. In addition, DDD also provides support for systems maintenance and evolution [11].

1.3. Agile Methods

One of the main reasons for using agile methods is the capability of keeping overtime and expenses in check while satisfying the needs of users [24]. Agile methods are most suitable for developing information systems in highly dynamic business environments. A dynamic business environment is reflected through rapidly changing requirements [17].

One characteristic of agile methods is the informal communication among stakeholders. Informal communication minimizes the necessity to document development progress, in contrast to conventional SDLC [25]. Furthermore, agile practices tend to determine specifications from user stories rather than from formal documents, and commercial modeling tools are seen as “tailored drawing environments” [24]. Despite the reduced need for documentation, the need for some documentation arises when there is a communication breakdown among stakeholders [3]. Communication breakdowns include factors such as “inability to scale the software, evolving the application over time, and inducting new member into the development team” or “personnel turnover, rapid changes to requirements, unavailability of appropriate customer representatives, or the application’s growing complexity” [3]. As such, the minimal documentation effort in agile development may jeopardize effective requirements determination.

Additionally, limitations of agile methods include limited support for 1) distributed systems development environments, 2) outsourcing to subcontractors, 3) building reusable artifacts, 4) large development teams, and 5) developing large, complex systems [23].

2. Research Method

2.1. Research Approach

This paper uses secondary data from a published case study [18] and grounded theory to derive concepts relevant to our base argument. Grounded theory involves obtaining “theory that was derived from data,

systematically gathered and analyzed through the research process” [21].

Orlikowski [19] states that grounded theory is a theory discovery approach that is inductive by nature, allowing researchers to develop a theoretical account while grounding the account based on empirical observations or data. In addition, grounded theory provides for development of a “context-based, process-oriented description and explanation of the phenomenon” [19].

Grounded theory consists of three phases, which are open coding, axial coding, and selective coding. Open coding focuses on conceptualizing or grouping similar items according to similar defined properties, defining categories, and developing categories in terms of their properties and dimensions, and proceeds by relating categories through hypotheses or statements of relationships. Axial coding is concerned with systematically developing and relating categories. Finally, selective coding integrates and refines the theory.

The data are extracted from a published case study, described in the following section. The data are explored using open coding to derive categories, axial coding to relate these categories, and selective coding to define theory based on data from the previous two phases.

2.2. Review of the Case Study

The case study [18], which examines the systems development project progress in conventional SDLC, is a published report based on action research conducted by Tetsuro Motoyama, Chief Research and Development Engineer of Advanced Imaging and Network Technologies at Ricoh, a company that primarily produces cameras and electronic office equipment. This case study was selected because it provides a complete narrative description of a conventional SDLC project that emphasizes documentation. Most other published case studies (e.g. Börjesson & Mathiassen [2], Kautz et al. [12], and Erdogmus [6]) are too condensed or were adapted to the use of specific research methodologies.

Motoyama’s case study began in 1993 by a small research group that explores various software technologies at Ricoh. The case study reviews two issues in the SDLC that force the author to improve the development process, involving sixteen projects divided into three stages. The three stages include problem recognition and initial response, process establishment and early trials, and continuous improvement.

The first stage involved familiarizing members with best practices of SDLC. The author specifies that

before introducing a new standard of development process, particularly the coding procedure, he familiarized himself with the new code inspection standard. The purpose of the first stage was therefore to motivate developers, particularly engineers, to write better code by making source code visible through inspection.

The second stage began three years after the initial stage. In this stage, the author considered the resolution to problems that occurred in the previous project. This stage saw another two projects, of which the first, referred to as J01, generated a development procedure based on ISO 9001 and lasted for about six months. The second project, J02, used the new development procedure from the previous project. In this stage, the author took several actions, which included separating designers and programmers, generating an integration-test plan from the technical requirements, encouraging each department member to create a list of tasks, implementing a risk officer and risk list, carefully reviewing design documents until inspection, encouraging the use of the Web to record activities progress, and recording and documenting problems.

The third stage focused on improving practices to overcome problems encountered during J02. The third stage started in 1999, precisely a year after J02. This stage was merely a continuation of the previous project. Activities at this stage included generating Design Document Guidelines, reviewing Document Change Requests, creating skeleton code before coding, conducting unit testing, tracking problems for improvement, improving the Web site that contains past-project information and relevant documents, initiating documents automation, and scheduling management.

3. Findings

3.1. Open Coding Phase

The findings are compiled concepts extracted from the case study by using grounded theory. As was mentioned, the open coding defines and develops categories. The emerging concepts are grouped in these categories. This phase specifically uncovers texts and exposes thoughts, ideas, and meanings contained therein [21]. During the axial coding phase, data that were fractured during open coding are reassembled to form major categories. Finally, these categories are integrated to form a larger theoretical scheme, which will be the theory behind this study.

Open coding is conducted to derive the following concepts. The initial impression that was found in this case study is that systems development projects regularly face many problems.

3.1.1. Stage 1: Problem Recognition and Initial Response. The objective of this stage was to refine the construction of source code based on established guidelines. To start out, Motoyama read some published guidelines, *Code Complete*¹. Motoyama's statement is as follow:

"To begin addressing our problems, I read *Code Complete* in late 1993. Then, I distributed the book to all the engineers in my department and instructed them to follow the techniques it describes."

This stage also included introducing two changes in the development process, which included inspecting code before check-in and effectively introducing the coding techniques in *Code Complete* as the standard for the department. These changes are described as follow:

"I also introduced two changes to our development process. One was to inspect code before check-in. On the basis of the guidelines in *Software Inspection* and *Software Inspection Process*, I simplified the inspection procedure to fit our small department. Second, I made the coding techniques in *Code Complete* our department's standard."

Furthermore, these changes resulted in better understanding of source code instruction that included code inspection to conduct early detection of design problems, implement easier error correction during integration, and complete partial code constructed by departed member(s).

"The code inspection resulted in the early detection of design problems, particularly among each module's interfaces. Another benefit was that correcting errors during integration became easier, because everyone in the project was familiar with each other's code. In addition, because of this familiarity, other engineers could complete a departing engineer's section of code."

Motoyama implies that the benefits of reviewing and inspecting code involve technical and social impacts. The members become more disciplined and code becomes better constructed. At this stage, documentation provides the benefit of early error detection.

3.1.2. Stage 2: Process Establishment and Early Trials. This stage involved a pilot project that began in 1996. Motoyama lists several problems encountered after the project's second iteration: coding before design, insufficient analysis of problems, a long design phase due to the lack of technical understanding and lack of architecture, repeated code segments, inspection of documents turning into design discussions, an unorderly sequence of code construction, difficult code integration due to compile and link errors, insufficient time to learn new materials, increased communication time to compensate for the lack of needed documents, and insufficient schedule tracking.

Motoyama and several members of the team proceeded with the next project, J01, which started in 1998. Motoyama states that this project successfully improved the development process by learning from the aforementioned problems and that the resulting development process abides to an acceptable standard of ISO 9001.

"In 1998, two junior engineers and I started a new project, which I'll call J01. This transitional project lasted only six months, but we had time to improve our development process. I generated a development procedure that covered a research project's entire life cycle and was based on ISO 9001."

The next project, J02, started in October 1998, about six months after J01. This project utilized the new development process with additional insights from an SDLC guideline document, *Software Project Survival Guide*².

The first insight included separating designers and programmers. Motoyama argues that this approach dramatically improves quality of the design documents in general. Motoyama explicitly cites the benefits of this approach, including improved inspection, easier recovery from designer's mistakes, significant improvement on quality of documents, and clear separation of prototype code and production code.

"In my approach, the designer of a section of code carefully examined that code during inspection, resulting in improved inspection. In addition, when the design contained errors, the programmer could more easily recover from the designer's mistakes because the programmer had participated in the design review and inspection.

¹ S. McConnell, *Code Complete*, Microsoft Press, 1993.

² S. McConnell, *Software Project Survival Guide*, Microsoft Press, 1998.

This approach had two other main advantages. First, because designers wrote the design documents to be readable and because programmers carefully evaluated others' design documents for coding, the documents' quality improved tremendously. Second, prototype code didn't creep into production code."

The early integration-test specification was to generate an integration-test plan based on the technical requirements while developers created a better design. This initiative benefited developers in that they became better prepared to address testing issues during code construction. Motoyama states specifically:

"In J02, I decided to generate an integration-test plan from the technical requirements while the engineers created the detailed designs...During programming, the programmers knew the test plan's contents, which helped them address testing issues during code construction."

Each team member regularly updated the task list at each development stage. The task list in general helped update the project schedule and thus increased the speed of the SDLC process. Motoyama states:

"We published the task lists on the Web, which motivated members to beat their estimated dates because everyone could see the project's progress. So, we finished J02 ahead of our estimated schedule. Later, when we had accumulated enough data, we could use the past task lists to estimate our next schedule. Because of our accumulated task lists, we're very comfortable with our schedule estimations."

The next initiative included creating a risk list and appointing a risk officer. Motoyama writes about the benefit of recording risks as follow:

"In this project, however, we immediately addressed the highest risk, letting us get back on schedule at the earliest possible time. So, we had sufficient time to recover from any unanticipated events and avoided any crippling problems."

Further, Motoyama implemented regular discussion meetings before inspection of defects.

"In J02, we added a discussion meeting before inspecting any documents to familiarize inspectors with design issues and let others contribute to our design's improvement."

Motoyama specified the use of the Web to generate HTML specification documents based on established processes, to record meetings, and to publish discussion and inspection results immediately after each meeting. Despite several problems, including design documents that were not uniform among designers and the large size of some documents, J02 provided the process basis for the next projects. Motoyama specifies his satisfaction with the second stage, particular J02 as follow:

"...we finished the project on time and completed the target functions.

J02 also established a basic practice for future projects—generating a document that described the project's problems and success."

3.1.3. Stage 3: Continuous Improvement. The goal of this stage was to improve the development process by overcoming problems encountered in projects in stage 2, particularly J02. The next project, J03, started in April 1999 and generated the Design Document Guidelines. Though these guidelines were not immediately accepted as department standards, they served as a basis for later improvements.

Motoyama mentions an initiative to use Document Change Request, which helps find errors in inspected and published documents. The pilot implementation of this form was not well received as members were reluctant and embarrassed to admit mistakes. It was not until J04 when Document Change Request is finally received and used throughout the remaining projects. The decision to fully utilize this form is as follow:

"Instead of hiding our mistakes, we decided to describe them to improve development. During J04, we increasingly used Document Change Requests. Also, we analyzed the reasons for the change requests at the project's conclusion as part of our Analysis of Software Development Process document. Most change requests concerned the detailed design documents."

The development team has had track records of problems encountered during the development process. These records are published as part of the project output documents. As part of the documents, the records are published automatically using the Web. Motoyama describes the intention to track problems as follow:

"Later, we tracked problems and successes through the Web instead of the whiteboard. We addressed these problems at the beginning of the

next project to improve our development process, documentation, and code conventions.”

Furthermore, automation is not only limited to tracking problems, but also for some parts of the development process.

3.2. Axial and Selective Coding Phases

This section analyzes the emerging concepts and theory from the previous section. The use of axial and selective coding encourages theory to emerge based on the previous concepts. Based on the following concepts, the theory uncovered in this study states that documentation, coordination, integration significantly reduce errors and accelerate systems development process. Figure 1 shows the theoretical model for this study.

Systems that have fewer errors and a reasonable development process according to its original schedule are the desired outcomes on almost all conventional SDLC projects. Any errors should be documented for future reference.

As was mentioned earlier, documentation significantly increases the successful outcomes of conventional SDLC, however, documentation alone is not sufficient to guarantee success. Good documentation requires coordination among development team members and stakeholders, and integrated tasks and responsibilities. Table 1 summarizes the concepts decoded in this study. The

following describes how the emerging concepts intertwine to develop the theory.

3.2.1. Learning from Established Documents. One of such essential concepts that emerge is that reviewing and learning from established documents significantly affects activities and actions, resulting in fewer mistakes, and outcomes resulting in fewer errors. Each team member is encouraged to review and learn from previous documents, which include development process guidelines and literature on development process. Best practices as lessons learned are applied to the actual development process. The application of best practices should be adjusted based on the needs and circumstances of the project. Learning from established documents significantly reduces errors and increases development speed, as members know what to do and what outcomes to expect.

3.2.2. Revisiting Previous Mistakes. This study finds that members of the team are often reluctant to admit mistakes that occur during the development process. Members are required to report and record not only progress and desired outcomes, but also mistakes, problems, and errors that significantly affect the development process. Only by doing so will all team members learn what to do to improve the development process and avoid project failure. Mistakes and errors should be documented for future reference.

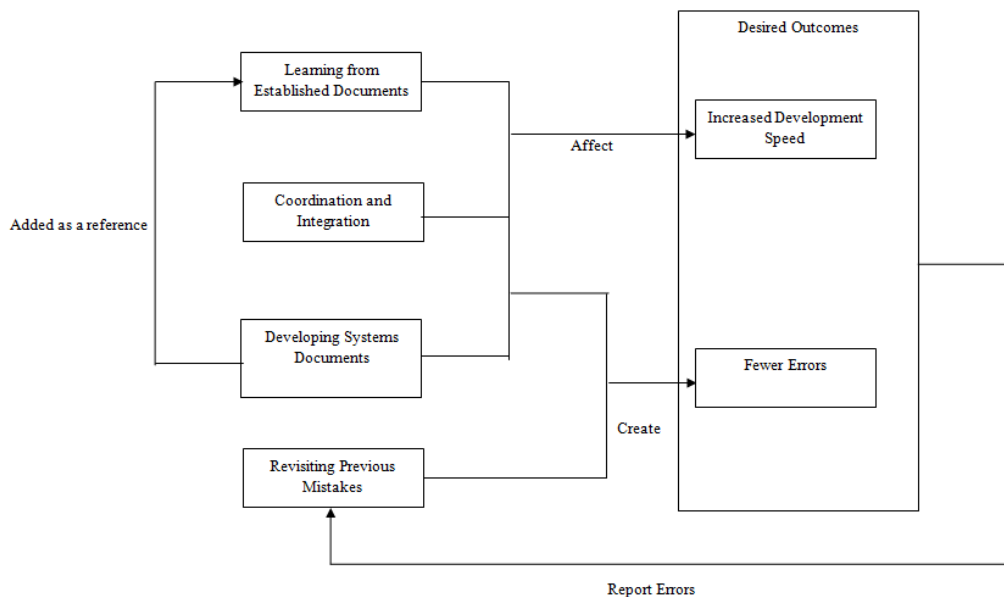


Figure 1 – Theoretical Model

3.2.3. Coordination and Integration. Not only the team members need to be managed, but also tasks and documents need to be coordinated and integrated. Team members meet regularly to discuss and report each other's progress and errors that may have occurred during the development process. Results of the meetings should also be documented for future reference.

Furthermore, responsibilities and tasks of each team member need to be clearly specified. The case study reports that members have overlapping tasks and responsibilities, sometimes resulting in prototype code creeping into the production code. Therefore, tasks should be clearly defined so that activities and responsibilities will not overlap. As such, clearly

defined activities and responsibilities greatly reduce errors and thus help increase development speed.

3.2.4. Developing Systems Documents. Document development is the most essential concept in this study. The use of documents is iterative in nature according to the model composed in this study. Previous SDLC documents can always be used as guidelines and references for upcoming projects.

Documenting involves recording any activity, mistake, and improvement in any phase of the development process. Documenting needs to be implemented not only for recording meetings. Documenting helps accelerate development process and reduces errors.

Table 1 – Concepts that Contribute to Defining Theory

Concepts	Descriptions
Learning from Established Documents	Includes reviewing documents from previous projects and other documents such as guidelines and literatures. Lessons of best practices are applied to the actual project.
Revisiting Previous Mistakes	Each member must admit mistakes that significantly affect development process. Mistakes should be documented as guidelines of what should not be repeated.
Coordination and Integration	Activities, documents, and personnel need to be coordinated and integrated efficiently. Members are encouraged to meet regularly and discuss any mistake and improvement occurred during development process. Tasks should be clearly defined so that activities will not overlap.
Developing Systems Documents	Includes regularly recording any activity, mistake, improvement at any phase of the development process. Documents should not be built during personnel meeting only.

4. Contributions and Conclusion

Very little research in SDLC has tried to show the superiority of conventional SDLC over agile methods. Much research has attempted just the opposite. Conventional SDLC is notoriously known for delayed deliverable outcomes, resulting in much increased project costs. This study does not seek to claim superiority of conventional SDLC; rather it identifies best practices for conventional SDLC through analysis of an existing case study. Most importantly, this study shows that SDLC documentation may indeed be an important contributing factor for successful system outcomes, in spite of what the reduced emphasis on documentation by agile practices may suggest. Thus,

the agile manifesto³ specifically values “working software over comprehensive documentation”, and Fruhling et al., for example, write that SDLC documentation needs to be “pared down to the essentials in creating environments that are more responsive to change” [8].

The contribution in terms of methodology is in the novel use of grounded theory to analyze a case study in SDLC research. Much research in SDLC involves the use of design science or quantitative methods in order to achieve objectivity. The use of a case study delineates the social perspective of SDLC, whereas SDLC includes not only systems and process, but also humans that control systems and process. The SDLC process can be considered a part of the world of

³ <http://agilemanifesto.org/>

humanly created meaning and thus is an integral part of the subject matter that the research is studying [14]. The use of an interpretive approach in researching SDLC shows that the SDLC process can also be seen as a social construction of reality.

This study seeks to address the use of comprehensive SDLC documentation that reduces failures in conventional SDLC projects, most notably delayed project outcomes and budget overruns. Lessons learned from this study are that SDLC documentation, members' coordination, and integrated tasks and documents can reduce such failures.

A limitation of this study is the use of an existing case study, rather than conducting field research. This may be the focus of future research, to validate the findings of the current study. Also of interest for future research may be investigating the effect of using more formal documentation as a means of communication in projects that otherwise make use of agile practices, rather than following conventional SDLC.

5. References

- [1] D. Avison and D. Fitzgerald, "Where Now for Development Methodologies?", *Communications of the ACM*, 46(1), 2003, pp. 79-82.
- [2] A. Börjesson and L. Mathiassen, "Improving Software Organizations: Agility Challenges and Implications", *Information Technology & People*, 18(4), 2005, pp. 359-382.
- [3] L. Cao and B. Ramesh, "Agile Requirements Engineering Practices: An Empirical Study", *IEEE Software*, 25(1), 2008, pp. 60-67.
- [4] N. Cerpa and J. Verner, "Prototyping: Some New Results", *Information and Software Technology*, 38(12), 1996, pp. 743-755.
- [5] G. Coleman and R. Verbruggen, "A Quality Software Process for Rapid Application Development", *Software Quality Journal*, 7(2), 1998, pp. 107-122.
- [6] H. Erdogmus, "What's Good Software, Anyway?", *IEEE Software*, 24(2), 2007, pp. 5-7.
- [7] J. Erickson, K. Lyytinen, and K. Siau, "Agile Modeling, Agile Software Development, and Extreme Programming: The State of Research", *Journal of Database Management*, 16(4), 2005, pp. 88-100.
- [8] A. Fruhling, P. McDonald, and C. Dunbar, "A Case Study: Introducing eXtreme Programming in a U.S. Government System Development Project", Proceedings of the 41st Hawaii International Conference on System Sciences, January 2008.
- [9] L. Garceau and E. Jancura, "Documentation and Training as a Systems Development Tool", *The CPA Journal*, 60(11), 1990, pp. 84-89.
- [10] S. Gavurin, "Where Does Prototyping Fit in IS Development?", *Journal of Systems Management*, 42(2), 1991, pp. 13-17.
- [11] Luqi, L. Zhang, V. Berzins, and Y. Qiao, "Documentation Driven Development for Complex Real-Time Systems", *IEEE Transactions on Software Engineering*, 30(12), 2004, pp. 936-952.
- [12] K. Kautz, B. Hansen, and D. Jacobsen, "The Utilization of Information Systems Development Methodologies in Practice", *Journal of Information Technology Cases and Applications*, 6(4), 2004, pp. 1-20.
- [13] M. Khalifa and J. Verner, "Drivers for Software Development Method Usage", *IEEE Transactions on Engineering Management*, 47(3), 2000, pp. 360-369.
- [14] A. S. Lee, "Researching MIS", in *Rethinking Management Information Systems: An Interdisciplinary Perspective*, New York: Oxford University Press, 1999, edited by W. L. Currie and B. Calliers, pp. 7-27.
- [15] K. Linberg, "Software Developer Perceptions about Software Project Failure: A Case Study", *The Journal of Systems and Software*, 49(2-3), 1999, pp. 177-192.
- [16] F. McCarey, "Agile Software Reuse Recommender", Proceedings of the 27th International Conference Software Engineering, May 2005.
- [17] P. Meso and R. Jain, "Agile Software Development: Adaptive Systems Principles and Best Practices", *Information Systems Management*, 23(3), 2006, pp. 16-30.
- [18] T. Motoyama, "Improving Software Development through Three Stages", *IEEE Software*, 23(5), 2006, pp. 81-87.
- [19] W. J. Orlikowski, "CASE Tools as Organizational Change: Investigating Incremental and Radical Changes in Systems Development", *MIS Quarterly*, 17(3), 1993, pp. 309-340.
- [20] J. D. Procaccino and J. M. Verner, "Software Project Managers and Project Success: An Exploratory Study", *The Journal of Systems and Software*, 76(11), 2006, pp. 1541-1551.
- [21] Strauss, A. and J. Corbin, *Basics of Qualitative Research: Techniques and Procedure for Developing Grounded Theory*, SAGE Publications, Thousand Oaks, CA, 1998.
- [22] B. Tsai, S. Stobart, N. Parrington, and B. Thompson, "Iterative Design and Testing Within the Software

Development Life Cycle”, *Software Quality Journal*, 6(4), 1997, pp. 295-309.

[23] D. Turk, R. France, and B. Rumpe, “Limitations of Agile Software Processes,” Proceedings of the 3rd International Conference on Extreme Programming and Flexible Processes in Software Engineering, May 2002.

[24] D. Turk, R. France, and B. Rumpe, “Assumptions Underlying Agile Software-Development Processes”, *Journal of Database Management*, 16(4), 2005, pp. 62-87.

[25] P. Wagstrom and J. Herbsleb, “Dependency Forecasting in the Distributed Agile Organization”, *Communications of the ACM*, 49(10), 2006, pp. 55-56.